



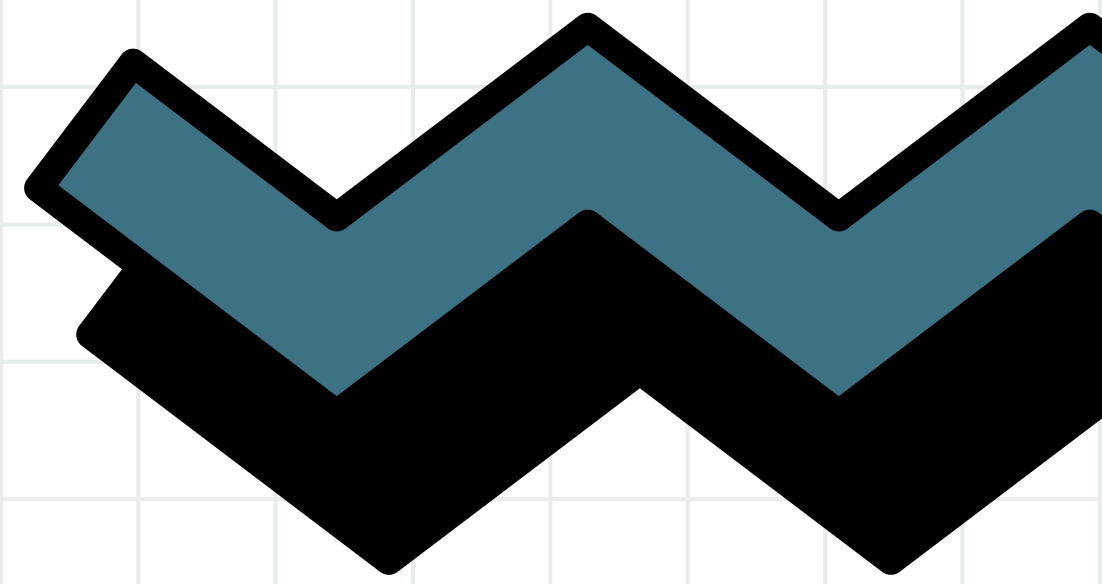
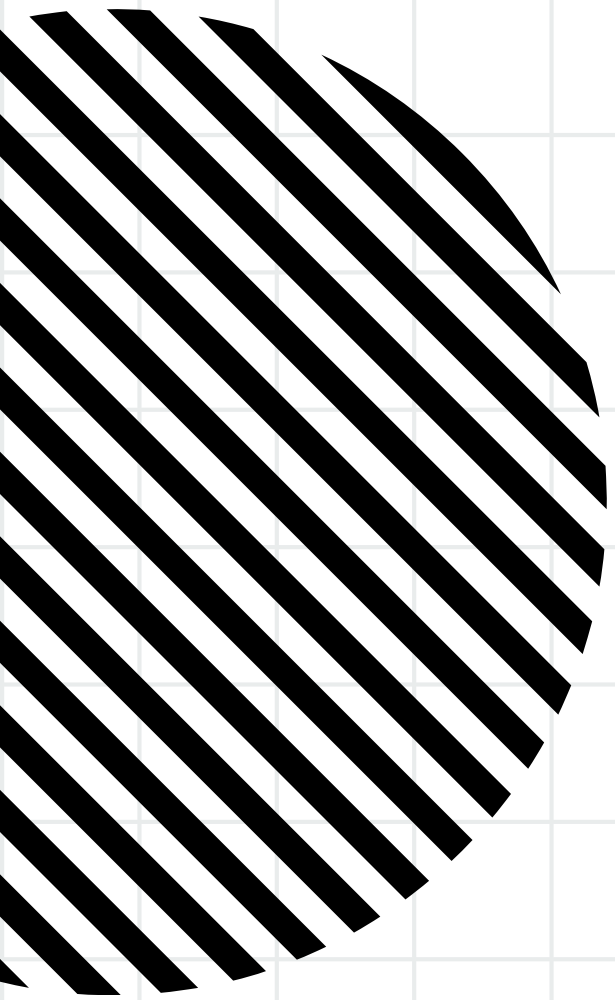
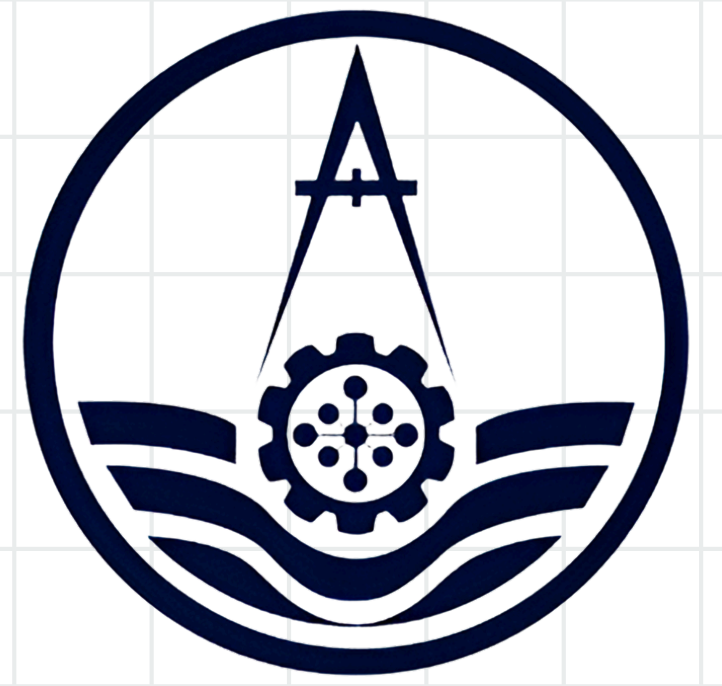
Universidad Autónoma de Campeche

Facultad de Ingenieria

Concurrencia y Paralelismo

Tecnologías de Software

5to "A"

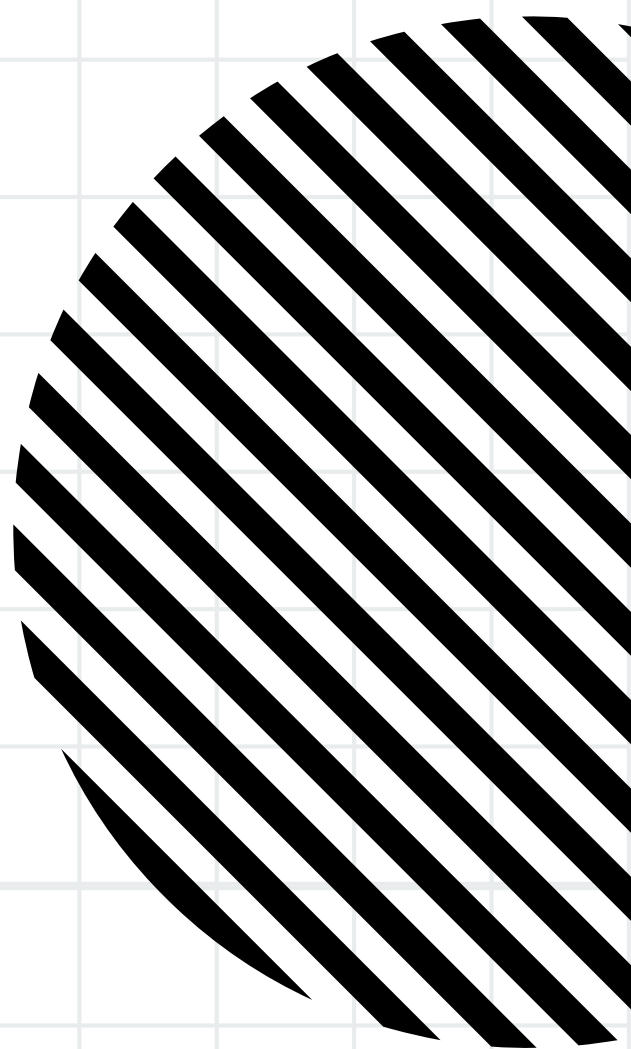
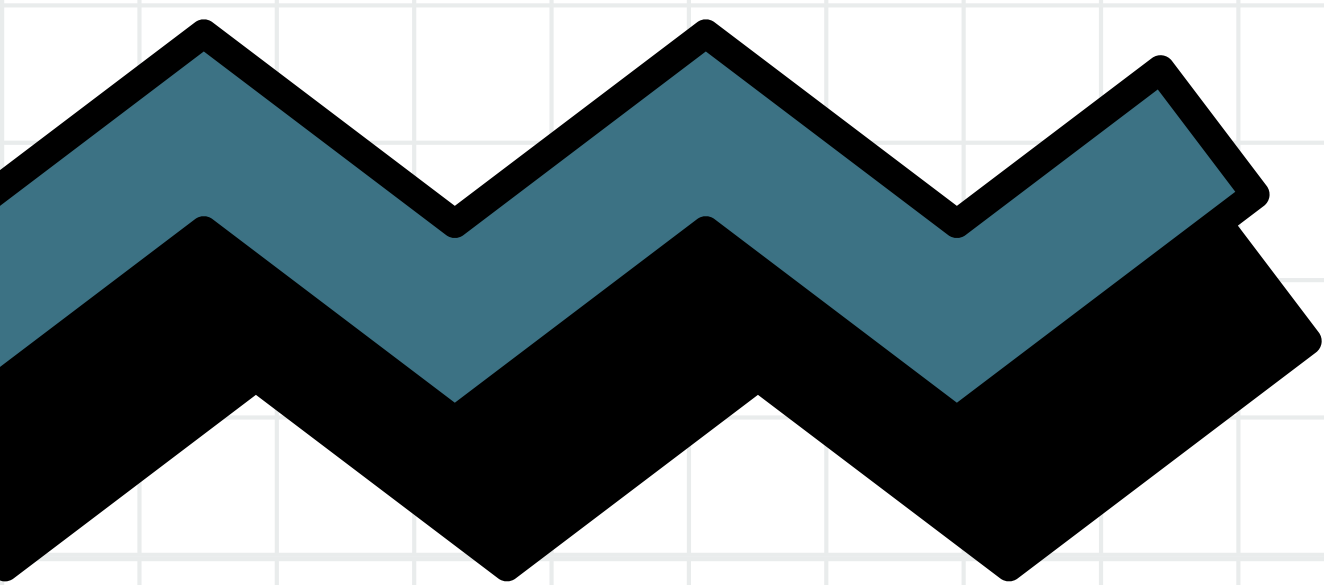


PRACTICA 03

Montecarlo

1er parcial

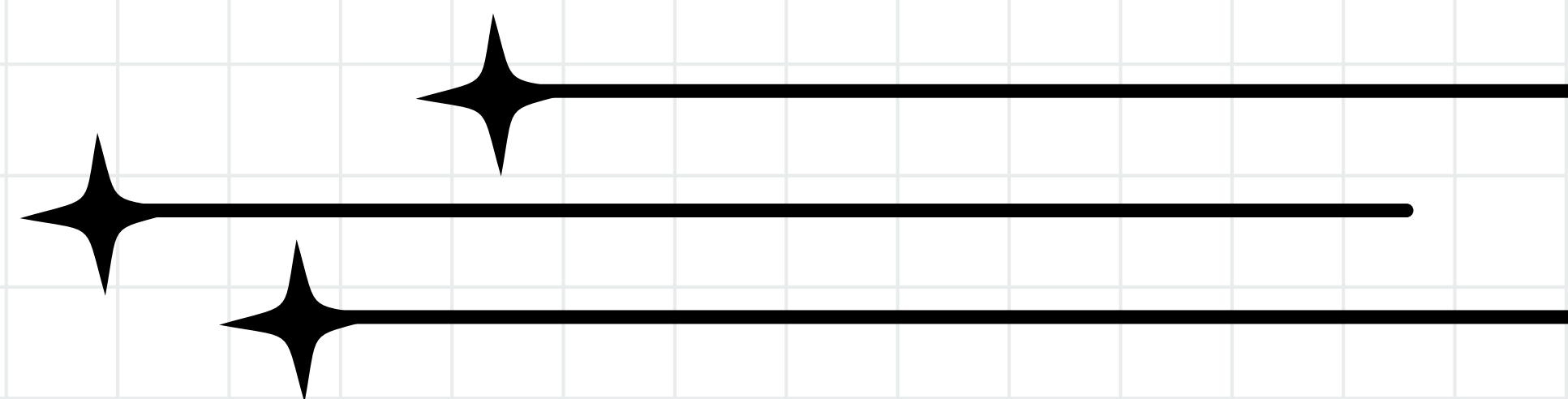
9 de Septiembre de 2026



Alumno: Victor Adrian Romero Minaya

Matricula: 67350

Docente: Sergio A Noh Puch



INTRODUCCIÓN

En el presente reporte se documenta el desarrollo y análisis de una práctica que combina el uso de hilos y semáforos para aproximar el valor de π mediante el método de Monte Carlo. El objetivo principal es implementar el pseudocódigo proporcionado, distribuyendo la generación de puntos aleatorios entre varios hilos y protegiendo con un semáforo el acceso a la variable compartida puntosDentro. A través de este ejercicio se busca observar cómo varía el tiempo de ejecución al modificar la cantidad de hilos, y entender si más hilos siempre significa mejor rendimiento.

DESARROLLO

Instrucción de la actividad: Implementar el pseudocódigo proporcionado y experimentar con ejecuciones.

1. **Implementación del pseudocódigo:** Para esta práctica, implementé el pseudocódigo proporcionado en un archivo llamado montecarlo.c. Cada hilo genera su propia porción de puntos aleatorios (x, y) entre -1 y 1 , dividiendo totalPuntos entre numHilos para repartir la carga de trabajo equitativamente. Por cada punto generado, se evalúa si cae dentro del círculo unitario con la condición $x^2 + y^2 \leq 1$; si es así, se incrementa un contador local (puntosLocal) dentro del propio hilo, evitando así tocar la variable compartida en cada iteración.

2. **Protección con semáforo:** Siguiendo el pseudocódigo, donde se indicaba `synchronized(lock) { puntosDentro++; }`, utilicé un semáforo (`sem_t`) inicializado en `1` mediante `sem_init(&sem, 0, 1)`, funcionando como un candado binario. Al terminar su ciclo de generación de puntos, cada hilo hace `sem_wait(&sem)` antes de sumar su contador local a la variable compartida puntosDentro, y `sem_post(&sem)` justo después, liberando el semáforo para el siguiente hilo. De esta forma, solo un hilo a la vez puede modificar la variable compartida, evitando una condición de carrera.

DESARROLLO:

3. **Medición del tiempo de ejecución:** Para medir el tiempo total, coloqué `clock_gettime(CLOCK_MONOTONIC, &inicio)` justo antes de crear los hilos, y `clock_gettime(CLOCK_MONOTONIC, &fin)` justo después de que todos terminan con `pthread_join`. Elegí `CLOCK_MONOTONIC` en lugar de la función `clock()` porque esta última mide tiempo de CPU consumido (que se puede sumar entre varios hilos), mientras que `CLOCK_MONOTONIC` mide el tiempo real transcurrido (wall-clock), que es lo que realmente necesitamos para comparar el rendimiento entre distintas cantidades de hilos.

4. **Pruebas con diferentes cantidades de hilos:** Siguiendo las indicaciones de la actividad, mantuve `totalPuntos = 1,000,000` fijo y ejecuté el programa modificando únicamente el número de hilos mediante un argumento de línea de comandos: `./montecarlo 2`, `./montecarlo 8` y `./montecarlo 16`. Para obtener una medición más confiable, corrí cada configuración 3 veces y promedié los tiempos obtenidos, ya que el tiempo de ejecución de una sola corrida puede variar ligeramente por factores externos del sistema operativo.

```
-----
Metodo Monte Carlo con semaforos
-----
Numero de hilos: 16
Puntos totales: 1000000
Puntos dentro del circulo: 786960
Valor aproximado de PI: 3.147840
Tiempo de ejecucion: 0.003523 segundos
```

```
Process finished with exit code 0
```

```
-----
Metodo Monte Carlo con semaforos
-----
Numero de hilos: 2
Puntos totales: 1000000
Puntos dentro del circulo: 785652
Valor aproximado de PI: 3.142608
Tiempo de ejecucion: 0.008871 segundos
```

```
Process finished with exit code 0
```

```
-----
Metodo Monte Carlo con semaforos
-----
Numero de hilos: 8
Puntos totales: 1000000
Puntos dentro del circulo: 786480
Valor aproximado de PI: 3.145920
Tiempo de ejecucion: 0.003291 segundos
```

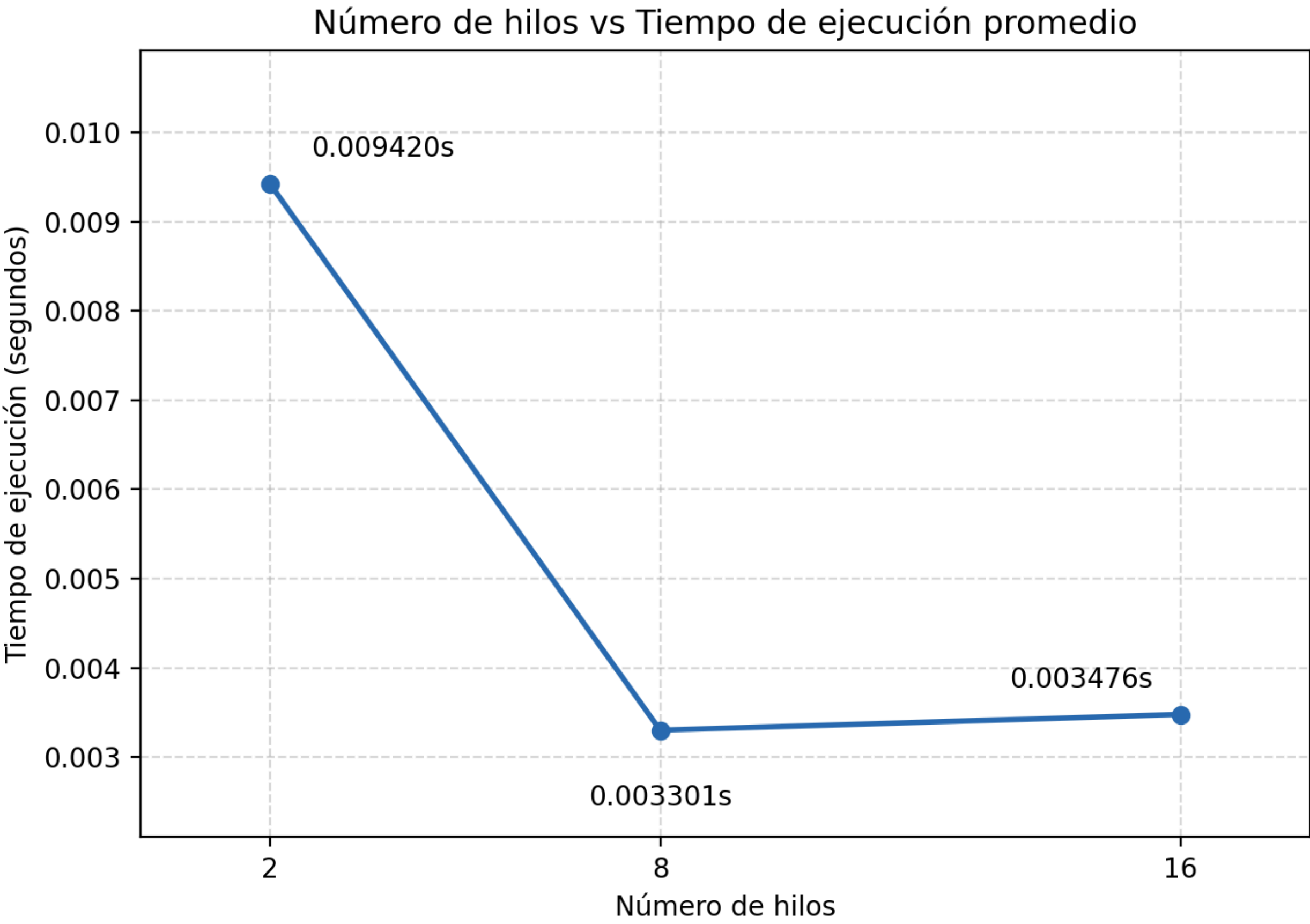
```
Process finished with exit code 0
```

DESARROLLO:

5. Tabla de resultados:

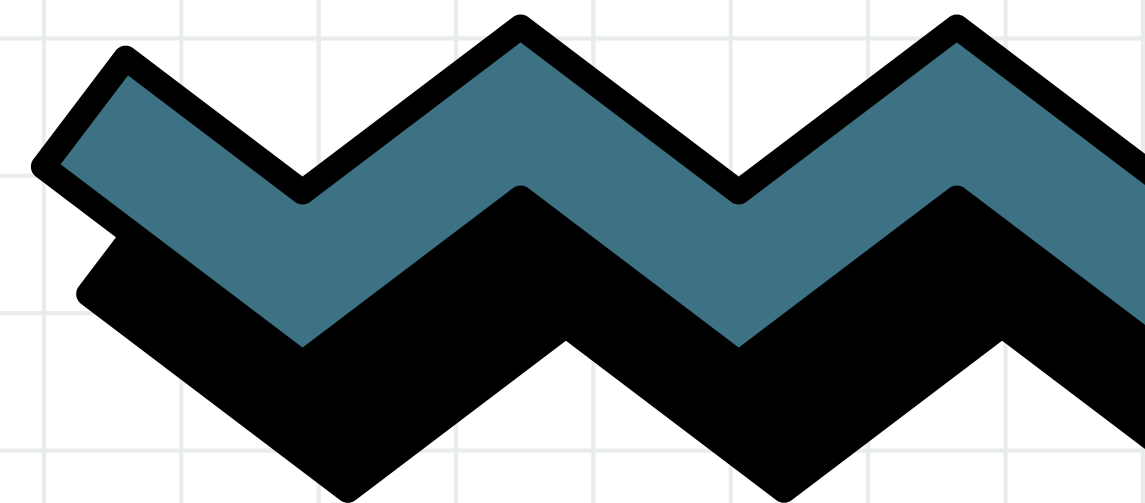
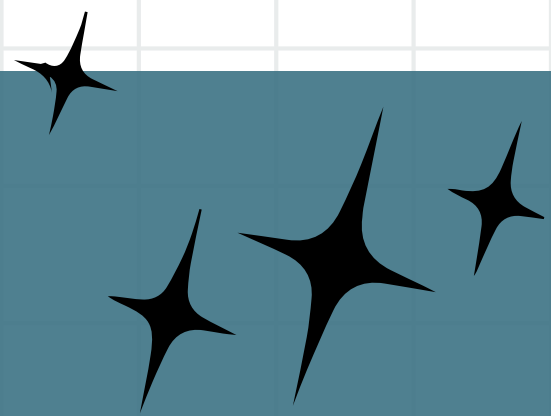
Número de hilos	Puntos dentro del círculo	π aproximado	Tiempo de ejecución promedio (s)
2	785,652	3.142608	0.009420
8	786,480	3.145920	0.003301
16	786,960	3.147840	0.003476

6. Gráfica de número de hilos vs. tiempo de ejecución:



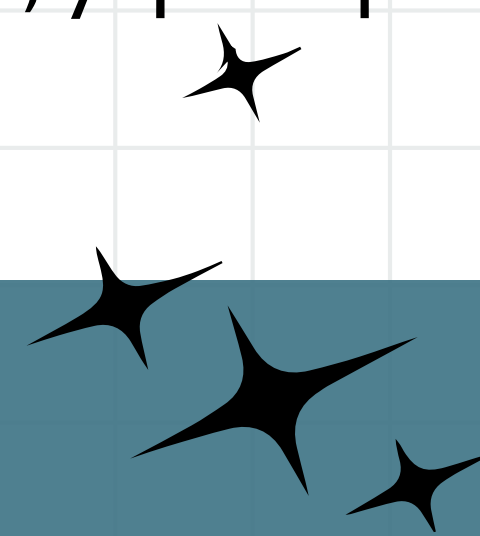
Como se observa en la gráfica, el tiempo de ejecución bajó drásticamente al pasar de 2 a 8 hilos (de 0.00942s a 0.0033s), pero al subir de 8 a 16 hilos el tiempo no mejoró más, sino que subió ligeramente (0.003476s).

CONCLUSION



Esta práctica me permitió comprobar de forma práctica el efecto de la paralelización sobre un problema de cómputo real. Usar un semáforo para proteger puntosDentro garantizó que, sin importar la cantidad de hilos, el resultado final del conteo siempre fuera correcto, evitando una condición de carrera. Al comparar los tiempos de ejecución promedio con 2, 8 y 16 hilos, la configuración de 8 hilos obtuvo el mejor rendimiento, siendo casi 3 veces más rápida que con 2 hilos. Sin embargo, subir a 16 hilos no trajo ninguna mejora adicional, e incluso resultó ligeramente más lento que con 8 hilos, lo cual sugiere que mi procesador cuenta con alrededor de 8 núcleos o hilos de ejecución lógicos disponibles, y que superar esa cantidad solo agrega overhead de administración de hilos sin beneficio real.

PREGUNTA DE ANÁLISIS



¿Aumentar la cantidad de hilos siempre mejora el rendimiento? No necesariamente, y mis propios resultados lo confirman. Al pasar de 2 a 8 hilos, el tiempo de ejecución se redujo notablemente (de 0.009420s a 0.003301s), porque el trabajo se repartió entre más núcleos disponibles del procesador y se aprovechó mejor el paralelismo real. Sin embargo, al subir a 16 hilos, el tiempo no mejoró — de hecho subió ligeramente a 0.003476s. Esto sucede porque, una vez que el número de hilos supera la cantidad de núcleos físicos (o hilos lógicos) que tiene el procesador, el sistema operativo ya no puede ejecutarlos verdaderamente en paralelo; en cambio, tiene que alternar entre ellos mediante cambios de contexto, lo que añade overhead sin aportar trabajo real adicional. Además, crear y gestionar cada hilo tiene un costo propio (asignación de recursos, pila de ejecución, etc.), por lo que a partir de cierto punto, agregar más hilos deja de ser beneficioso y puede incluso empeorar ligeramente el rendimiento.



¡Gracias por leer!